Lecture 3

Subsystem Design

Subsystem Design

Outlines

- Modeling Subsystems Design
- Subsystem Interface Dependency Viewpoint
- The Layered Subsystem Viewpoint
- Mapping Subsystems and Layers to Implementation
  - Subsystems, layers, and build trees
  - Subsystems and components

Subsystem Design

- In software architecture, a **subsystem** is a well-defined part of a larger software system.
- It represents a logical grouping of related elements that work together to provide a specific set of functionalities.
- In modeling diagrams (like UML), a subsystem is typically visualized as a stereotyped package.
- **Package**: A collection of model elements (classes, interfaces, components, nodes, collaborations, use cases, and other packages)
    - All model elements (except for the top-level system package) must be contained within a package.
- A **system** is shown graphically as a stereotyped package, and is a representation of the entire scope of the development effort.
- A **subsystem** is a part of the system and, as with the system, is shown as a stereotyped package.

Subsystem Design...

- Key design principles for effective subsystem creation and **maintainability** :
  - **high cohesion:** Elements within a subsystem should be strongly related, focusing on a single, well-defined purpose
  - **low coupling:** Subsystems should have minimal dependencies on each other.
    - *This promotes modularity and maintainability*
- Subsystems are the **lowest-level entity** managed by software architecture teams for:
  - Interface management
  - Design documentation
  - Testing (automated test suites, defect resolution)
  - Delivery units (source code, documentation, test reports other delivery products).

Subsystem Design...

- Another technique to help abstract a **system design** is the use of **layers**.
- **Layers** :
  - Decompose system functions into **hierarchical groups**
  - **higher layers** depend on functions of the **lower layer**.

Example: In an e-commerce system, subsystems might include:

- User management subsystem
- Product catalog subsystem
- Order processing subsystem
- Payment processing subsystem

Subsystem Design...

Modeling Subsystems, Inter faces, and Layers

- **Modularity:** Breaking down a system into independent, interchangeable modules.
  - This improves maintainability and allows for easier updates.
- **Subsystems** and **layers** are two fundamental tools for structuring large-scale systems into smaller, more manageable parts.
- A **subsystem** groups related elements to enable independent design, development, management, and testing.
- The decomposition of software system leads to the creation of dependencies between subsystems.

Subsystem Design...

Modeling Subsystems, Inter faces, and Layers

Interface

- A contract that specifies how two subsystems or components interact each other.
- Interfaces promote loose coupling and allow for independent development.
  - **Importance:**
    - Abstraction: Hides internal implementation details.
    - Standardization: Ensures consistent communication.
    - Flexibility: Allows for independent development and replacement of subsystems.

Subsystem Design...

Modeling Subsystems, Inter faces, and Layers

Layers:

- Provide a hierarchical structure, organizing subsystems and components to manage complexity and promote modularity.
- Common Layers
  - *Presentation Layer:* Handles user interaction (e.g., UI, web pages).
  - *Business Logic Layer:* Implements the core business rules and logic.
  - *Data Access Layer:* Handles database interactions and data persistence.

- *Dependencies:* Represent the relationships between subsystems and layers, indicating how changes in one part of the system might affect others.

Subsystem Design...

Subsystem Interface Dependency Viewpoint

- This viewpoint focuses on the relationships and dependencies between subsystems through their interfaces.

External Interface Design

- Defining the interfaces that a subsystem exposes to other subsystems or external systems.
- Key Consideration: security, performance, and usability.
- An Interactions with **third-party systems** (e.g., APIs, databases).
- Designing a RESTful API for mobile applications to access the system.

Example: Billing subsystem depending on a payment gateway (external).

Subsystem Design...

Subsystem Interface Dependency Viewpoint

Internal Interface Design

- Specifies contracts between **subsystems within the same system**.
- Key Consideration: modularity (Loose coupling) within the subsystem.
- Defining message queues for asynchronous communication between the subsystems.

Example: Billing subsystem calling the Inventory subsystem (internal).

Graphical User Interface (GUI) Design

- Interfaces for user interaction (e.g., web/mobile frontends).
- Key Consideration: usability, accessibility, and user experience.
- Designing a responsive web interface for the system that adapts to different screen sizes.

Example: Checkout page (depends on billing logic)

Subsystem Design...

Subsystem Interface Dependency Viewpoint

Example: Monthly Billing Use Case subsystem interface dependency diagram:

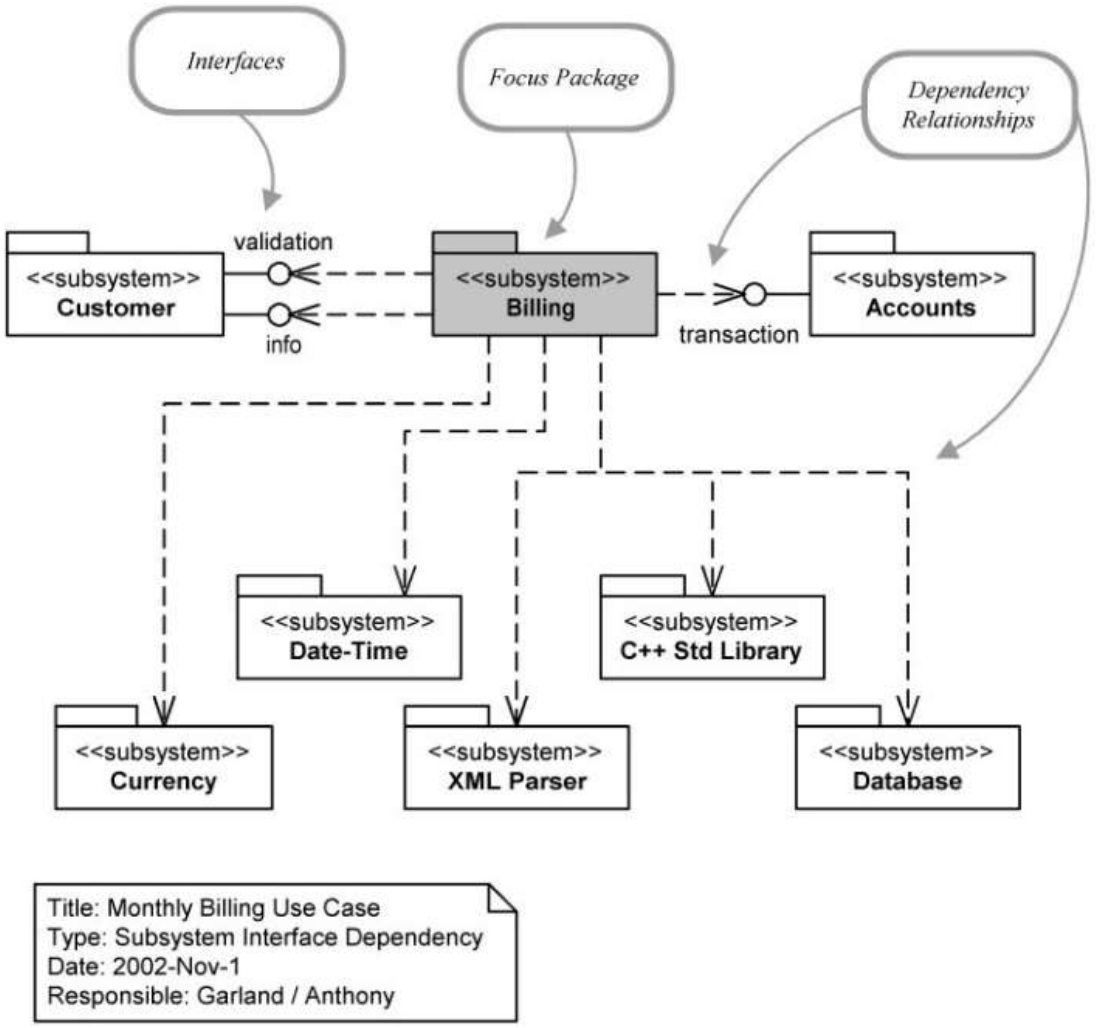
Subsystems and Relationships:

1. External Interfaces

- **Validation**: Ensures data correctness
- **Customer**: Manages customer profiles and interactions.

2. Focus Package

- **"Billing"** subsystem (central focus Package)



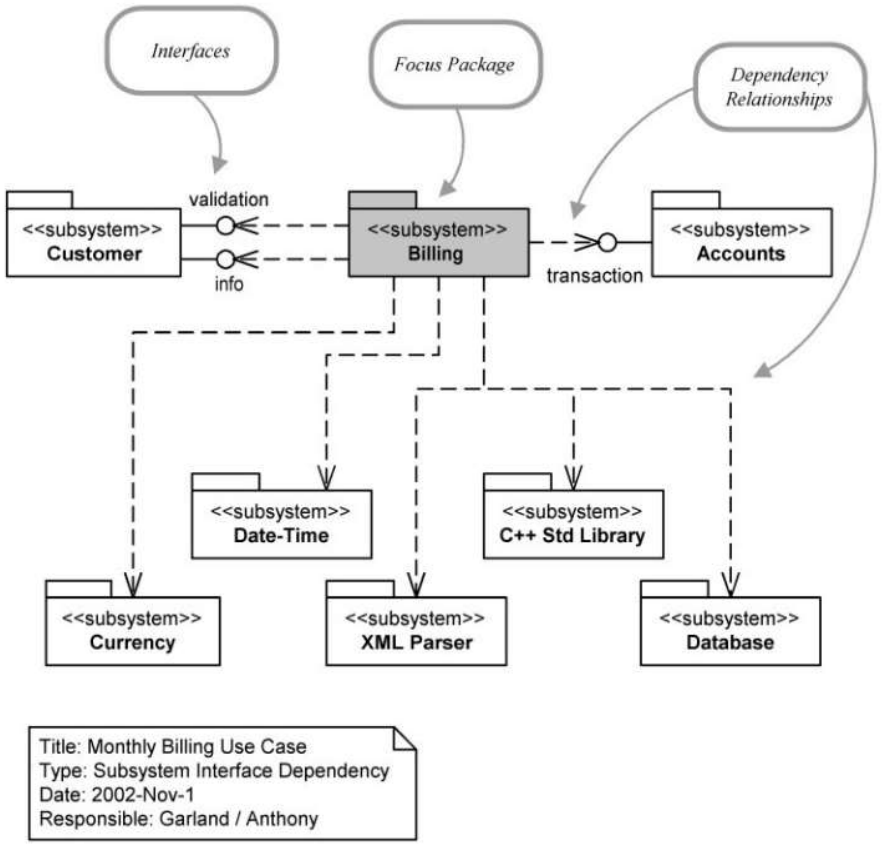
Subsystem Interface Dependency View

Subsystem Design...

Subsystem Interface Dependency Viewpoint

3. **Dependency Relationships :**

- **Accounts**: Tracks customer accounts/balances.
- **Date-Time**: Handles billing cycles, due dates, and timestamps.
- **XML Parser**: Processes data formats (e.g., invoices, exports).
- **Database**: Stores all transactional/account data.
- **Currency :** Handles multi-currency conversions, formatting, and exchange rates
  - **Dependencies**:
    - **Date-Time** for rate validity periods.
    - **Database** to fetch/store exchange rates.
  - **Used by Billing/Accounts** to support international transactions.



Subsystem Interface Dependency View

Subsystem Design...

Subsystem Interface Dependency Viewpoint

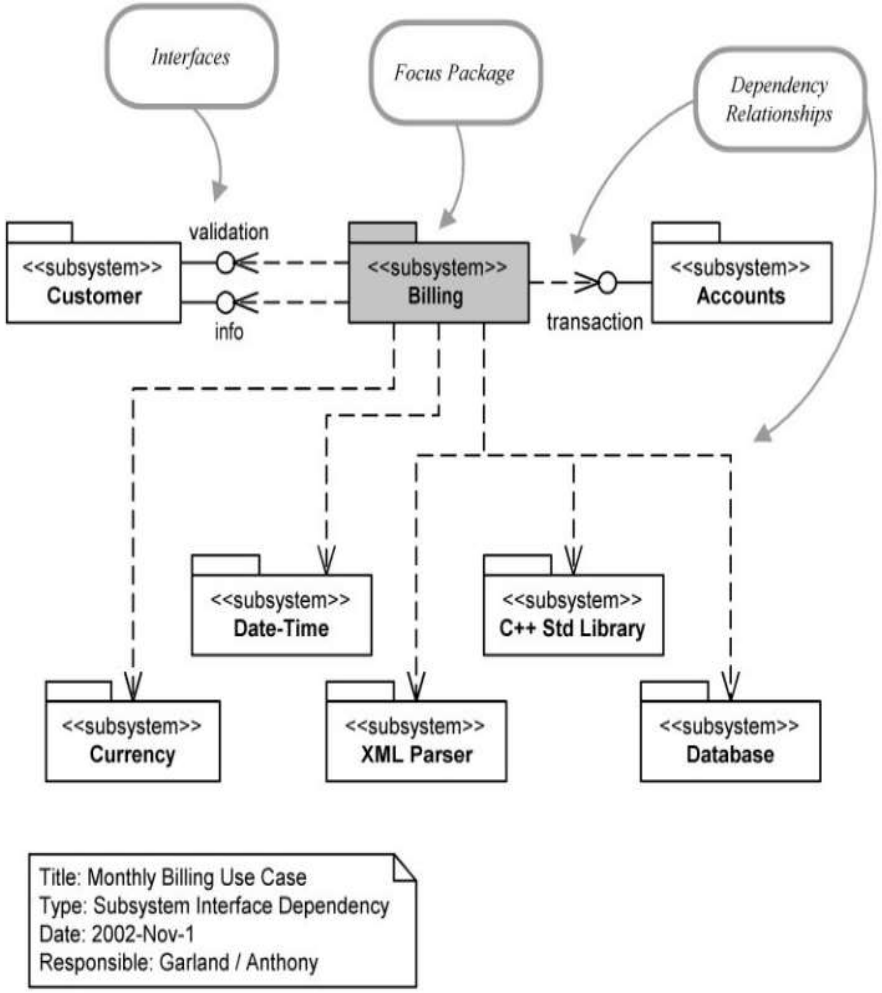
Workflow for Monthly Billing

Data Validation

- The **Customer** subsystem sends customer details (e.g., name, account ID) to the **Validation** interface.
- **IValidation** checks for errors (e.g., missing/invalid data) and approves the data for billing.

Fetch Account Details

- **Billing** subsystem requests account balances, payment history, and tariffs from the **Accounts** subsystem.



Subsystem Interface Dependency View

Subsystem Design...

Subsystem Interface Dependency Viewpoint

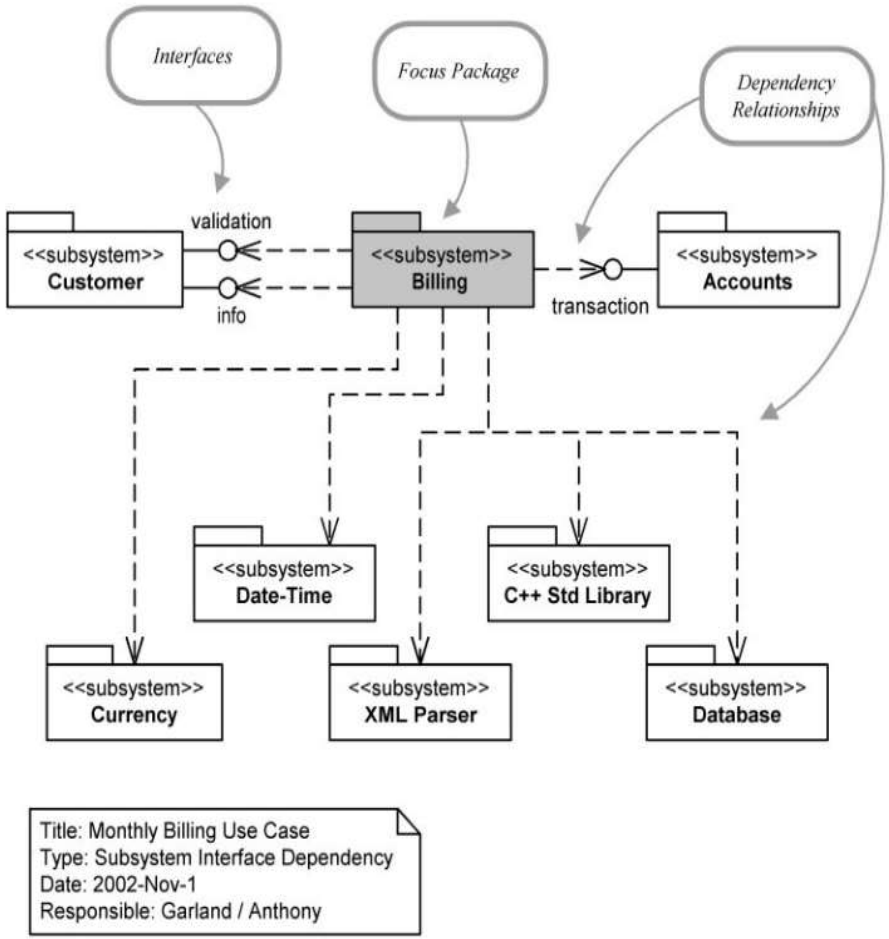
Workflow for Monthly Billing..

Billing Calculation

- **Determine Billing Cycle**
  - **Billing** uses the **Date-Time** subsystem to:
    - Identify the current billing period
    - Apply due dates/penalties if payments are overdue.
- **Generate Invoice**
  - **Billing** calculates charges (e.g., usage fees, subscriptions).
  - **XML Parser** formats the invoice into a structured document (e.g., XML for integration with other systems).

Data Persistence & Output

- **Save Records & Deliver Output**
  - Transaction details and invoice copies are stored in



Subsystem Interface Dependency View

Subsystem Design...

Enhancing the Subsystem Dependency Views with layers

- Layering is a successful design strategy for structuring software systems.
- Layers can simplify complex views by suppressing detail and focusing on specific scenarios.
- Hiding interface dependencies reduces the "information level" in a view.
- **Simplifies complexity** by grouping subsystems into layers.

Benefits

- **Maintainability**: (loose coupling).
- **Scalability**: Easy to add new subsystems
- **Testability**: Layers can be mocked/stubbed for unit testing.

Subsystem Design...

Enhancing the Subsystem Dependency Views with layers

Layered Architecture for Monthly Billing System

1. **Domain Layer** (Business Logic)

   Customer, Billing, Accounts

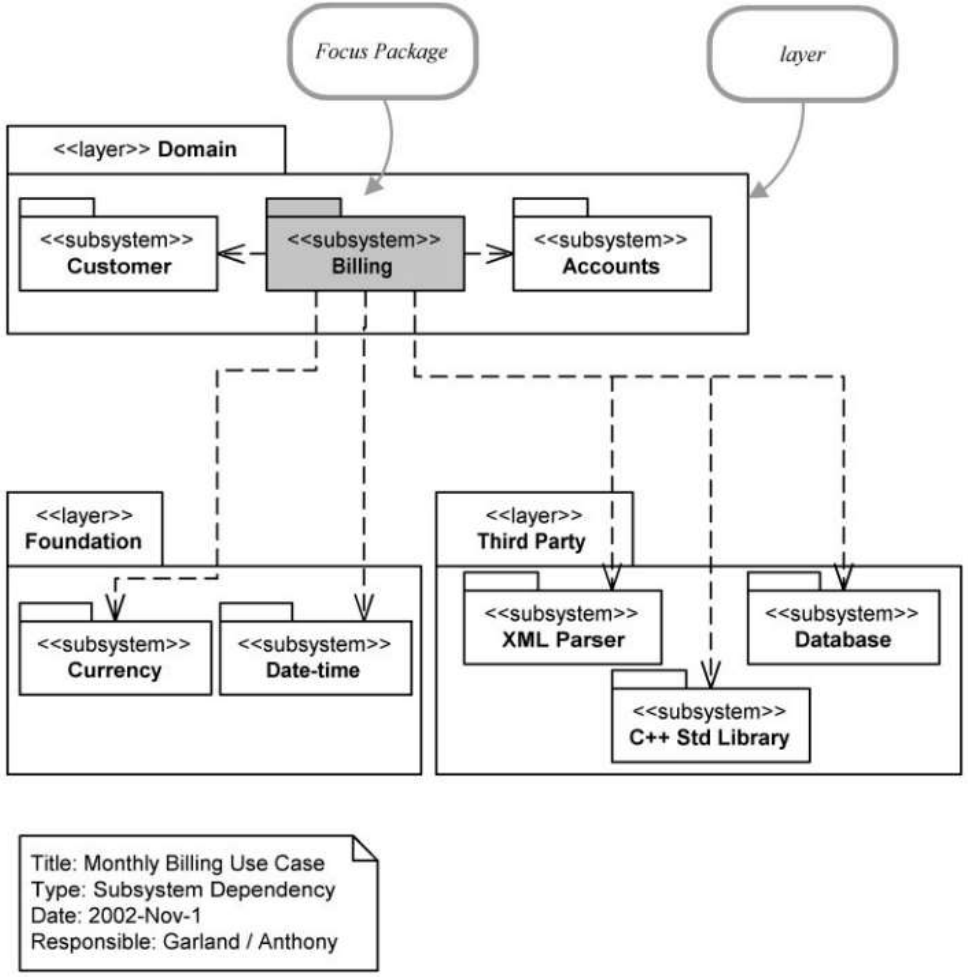
2. **Foundation Layer** (Supporting Utilities)

   Currency, Date-Time

3. **Third-Party Layer** (External Integrations)

   XML Parser, Database, C++ Standard Library (Std)

- The current diagram has no Top/application layer focuses on *internal* subsystem dependencies rather than user-facing interfaces
- This architecture ensures a clean separation of concerns, making the system adaptable to future

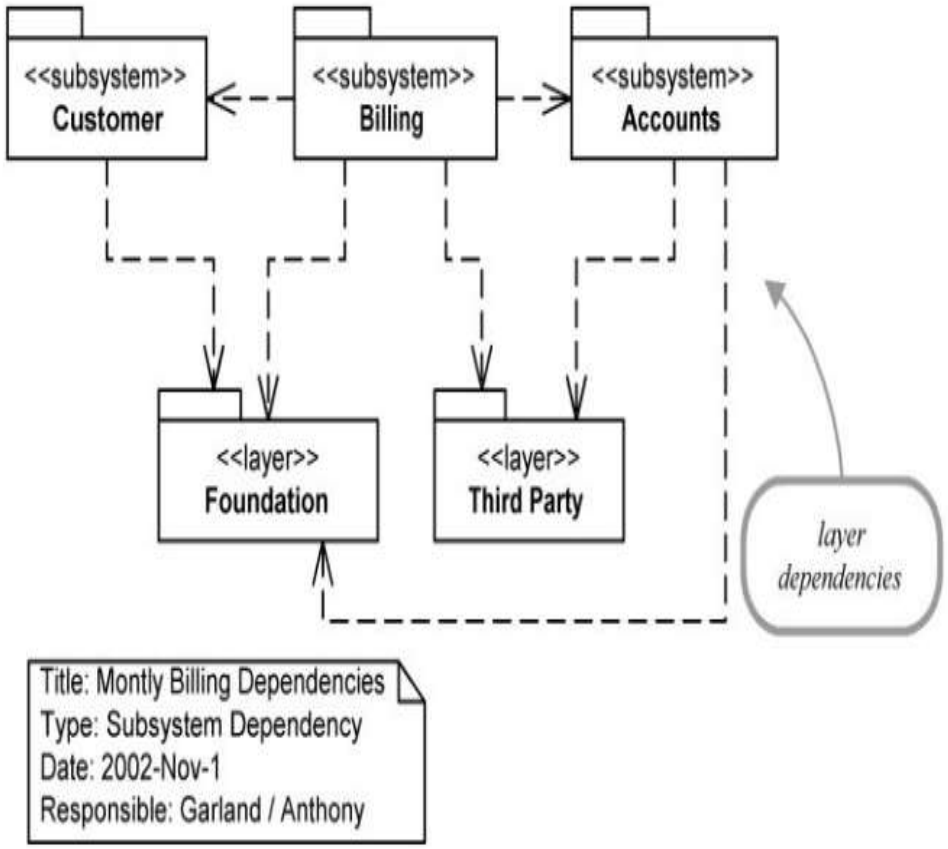


Subsystem Dependencies with layers

Subsystem Design...

Top-level Dependencies

- Describes the high-level relationships between major subsystems and layers.
- This helps to understand the overall architecture of the system.
- A **top-level** version of the Subsystem Interface Dependency View describe the '**organizational interfaces**'.
- This allows software architects
  - To define development team boundaries,
  - To communicate and negotiate interfaces between development teams, and
  - To communicate with project management.
- The view typically omits infrastructure (third party) and off-the-shelf subsystems (currency).



Subsystem Interface Dependency View – use case focused, with layers

Subsystem Design...

The Layered Subsystem Viewpoint

- Organizes subsystems into layers in order to provide a single view that represents the entire architecture.
- Easy-to-understand views for non-software experts and adding dependency semantics for the technical teams.

Layer Hierarchy:

1. ***Lowest/ Third-party***: Third-party (databases, libraries).
2. ***Middle/Project utilities*** : Project utilities (shared tools, logging, security).
3. ***Domain/Domain-specific***: Business logic (e.g., OrderProcessing).
4. ***Top/Application:*** Application-specific/user-facing features (e.g., User Dashboard, Checkout GUI).

Purpose :

- Clear for stakeholders.
- Enforces dependency rules (e.g., domain layer cannot bypass utilities).

Subsystem Design...

The Layered Subsystem Viewpoint

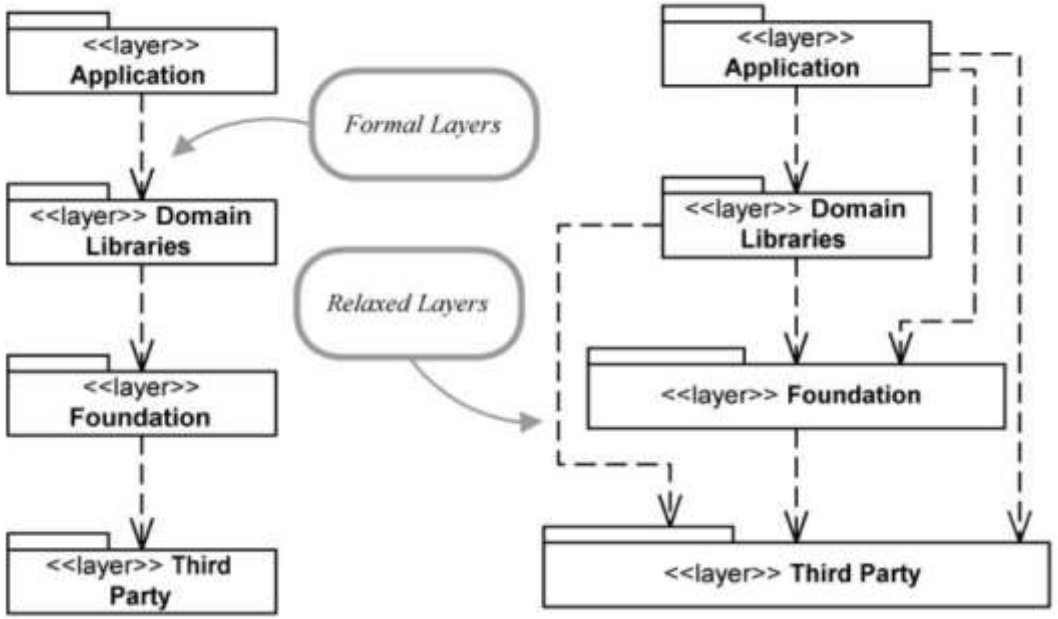
layered architecture with both **formal** and **relaxed** layer

Formal Layers (Strict Hierarchy)

These layers enforce strict dependency rules (dependencies only flow downward):

•**Application → Domain[Libraries] → Foundation → Third Party**

- Example:
  - *Application* (UI/API endpoints) depends on *Domain* (business logic).
  - *Domain* depends on *Foundation* (shared utilities).
- No upward or circular dependencies (e.g., *Foundation* cannot depend on *Application*).
- Used in High-criticality systems (e.g., banking) where



Layering semantics – formal and relaxed

Subsystem Design...

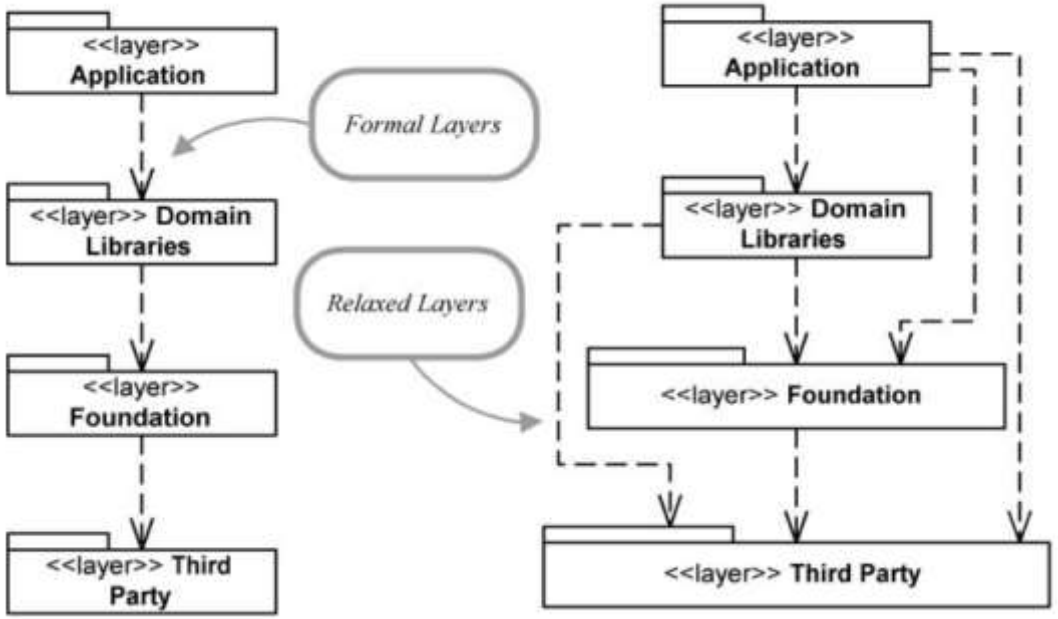
The Layered Subsystem Viewpoint

layered architecture *with both* ***formal*** *and* ***relaxed*** *layer*

Relaxed Layers (Flexible Hierarchy)

These layers allow pragmatic flexibility, possibly with controlled exceptions:

- Example:
  - *Domain Libraries* might bypass to directly use *Foundation* (e.g., for performance).
  - *Third Party* could be accessed by any layer (e.g., a logging library used globally).
- Dependencies may skip layers or be bidirectional if justified (e.g., via interfaces
- Used in Rapidly evolving systems (e.g., startups) where pragmatism outweighs strict rules.

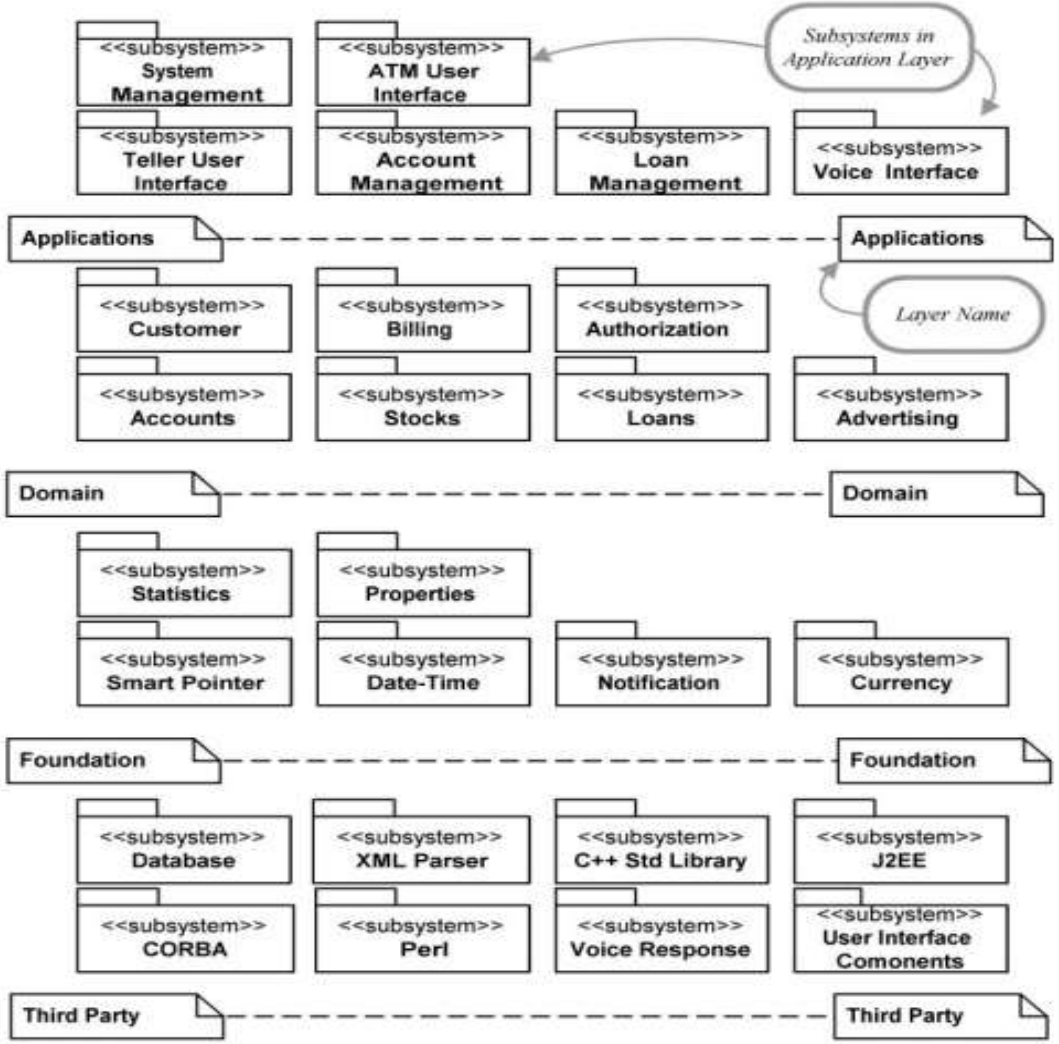


Layering semantics – formal and relaxed

The Layered Subsystem Viewpoint

Typical layers include:

- *The lowest layer:* Third-party subsystems (e.g., database, UI libraries).
- *The second layer:* Project-maintained subsystems for common programming tasks.
- *The third layer:* Components specific to the problem domain.
- *The top layer:* Application subsystems, often extending lower-layer functionality.



high-level overview of how subsystems are organized within a layered architectural pattern.

Mapping Subsystems and Layers to Implementation

Subsystems, layers, and build trees

- Subsystems, layers, and build trees are all concepts used in software architecture to organize and structure software systems.
- A build tree is a hierarchical structure that represents the dependencies between the different components of the system.
- The build tree is used to compile the source code into executable code, and ensures that all of the necessary components are included in the final build.
- A **subsystem** in the design is realized as a subdirectory of the same name in the implementation directory tree.
- A subsystem directory should contain common sub directories to manage **Source code**, **Test suits** and **Documentation**
- If layers are used in the architecture, they are also reflected as a directory in the implementation.

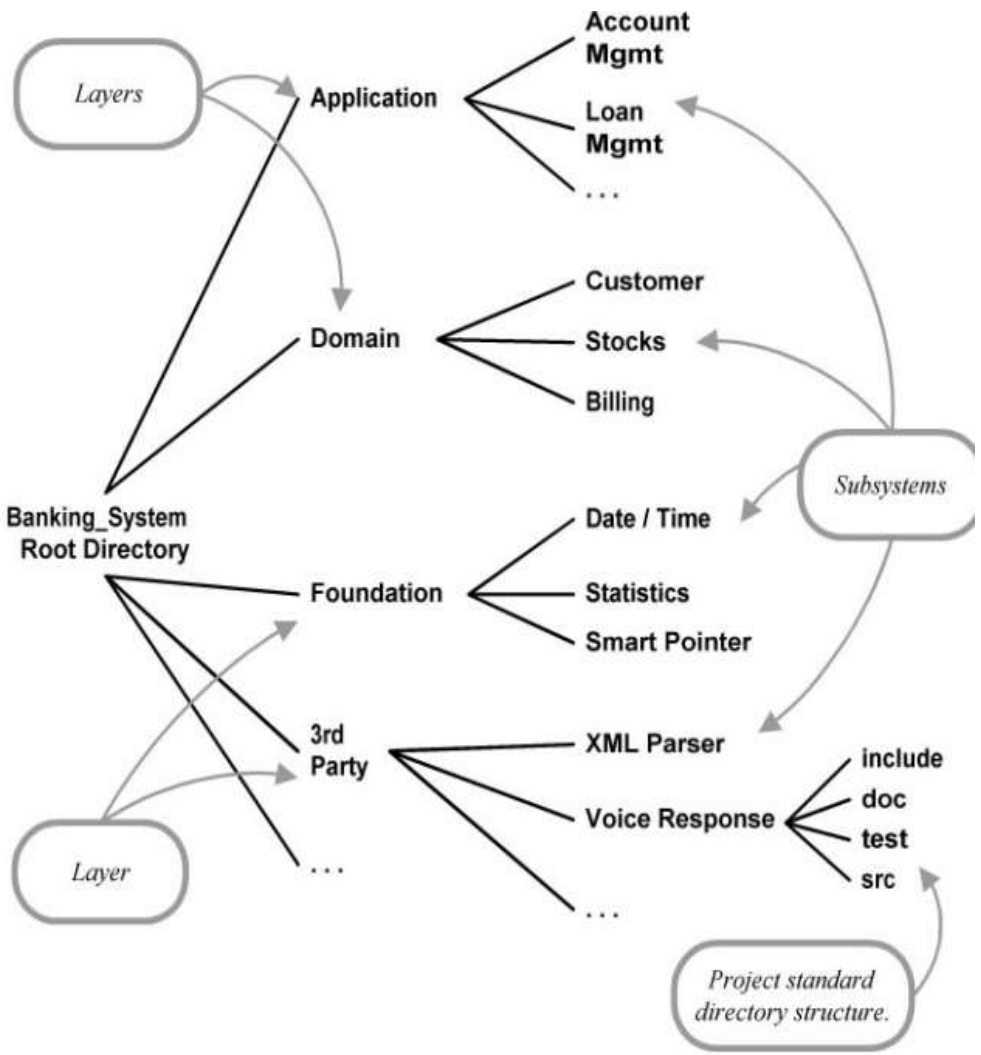
Mapping Subsystems and Layers to Implementation

Subsystems, layers, and build trees

- **Directory Structure**:
  - Subsystems → Subdirectories (e.g., /billing/).
  - Layers → Parent directories (e.g., /third-party/, /domain/).
- **Build Tree**:
  - Hierarchical compilation dependencies (e.g., domain/ must compile before application/).
- With the directory date can be written in the source code as

  #include "foundation/date-time/date.hpp"
- In short it can be written (if configured)

  #include "date.hpp"
- Standard directory structure used for each subsystem such as **include, doc, test ,src**.

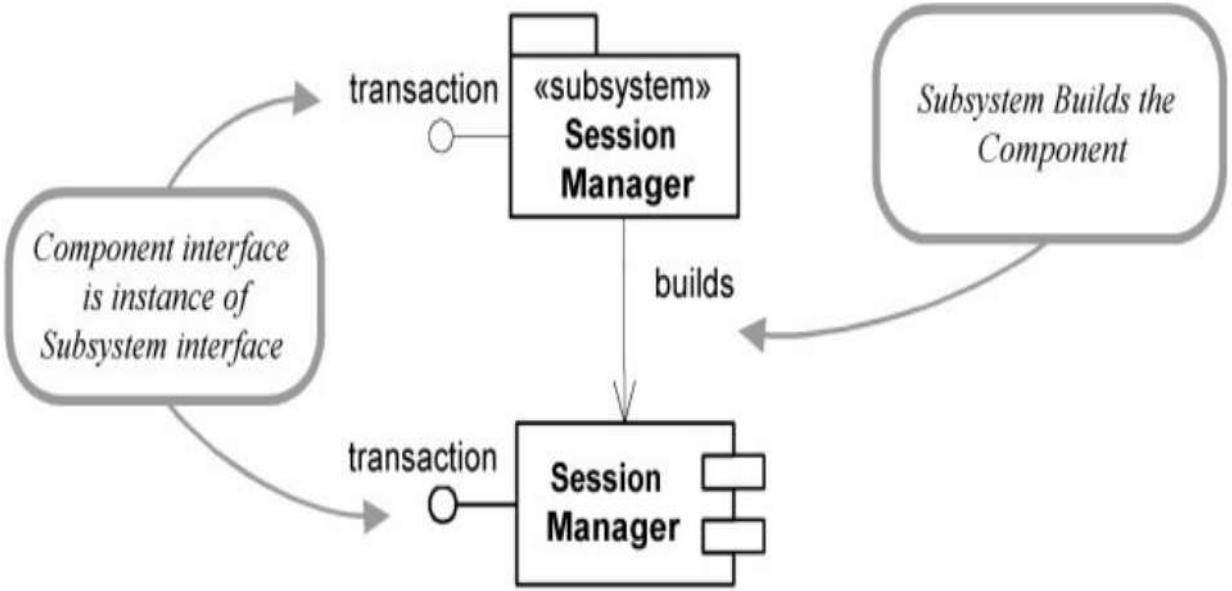


Mapping layers and subsystems to a directory structure

Mapping Subsystems and Layers to Implementation

Subsystems and components

- Subsystems and components are related but distinct:
  - A subsystem is a design-time entity.
  - A component is a runtime instance of a subsystem.
- When we build, install and execute a subsystem a running component will be created.



Subsystem and component conceptual relationship

Key Takeaways

Subsystems group related functionalities for **modularity** and **maintainability**.

Layers provide a **hierarchical structure** to manage complexity.

Interfaces define **how subsystems communicate**, promoting loose coupling.

Formal vs. Relaxed Layering offers flexibility based on project needs.

Directory structures and **build trees** map design to implementation.

Thanks!